

Department of Computer Engineering & Computer Science

M.S. THESIS RESEARCH BRIEF · FOR PROPOSED COMMITTEE MEMBERS

Automated UML Dataset Generation from Natural-Language Requirements with Multimodal Verification for Software Design

Student: Dipak Yadav, M.S. Computer Science

Thesis chair: Yutong Zhao, Ph.D., Assistant Professor, CECS

Email: dipak.yadav5501@gmail.com · Repository: github.com/dipak5501/uml-generation-pipeline

1. Problem and motivation

Unified Modeling Language (UML) is a standard medium for communicating software architecture, but constructing diagrams by hand is slow, inconsistent, and hard to scale when requirements are informal or frequently revised. Large language models can draft diagrams from text, yet a diagram that compiles is not necessarily a correct design: it may omit entities, misuse relationships, or look valid while remaining conceptually misleading. Prior work has emphasized generation more than verification, and existing UML datasets are typically small, narrowly typed, or scored only by syntax checks or a single model. This thesis treats **verification as a first-class stage** of automated UML construction.

2. Research objective and questions

The objective is to generate design-phase UML from natural-language requirements at dataset scale, then validate each artifact with a vision-language model (VLM) ensemble that can be compared with expert ratings. Four families are studied: **class, object, component, and package**. Three research questions guide the evaluation:

RQ1. How effectively can a decomposed LLM pipeline generate syntactically and semantically valid UML artifacts from natural-language requirements at scale?

RQ2. To what degree does multimodal ensemble verification correlate with expert human evaluation of the generated diagrams?

RQ3. How does accuracy vary across UML diagram families, and what failure patterns explain the differences?

3. Method: three-stage pipeline

The framework assigns each model a narrower job so failures can be isolated at stage boundaries, and the PlantUML generator never sees raw unprocessed requirement text.

Stage 1 — Technical specification (LLaMA 3.2-1B-Instruct). A free-form requirement becomes structured JSON: entities and attributes, relationships (association, aggregation, composition, dependency, realization), containment, and diagram-type constraints. A 1B instruction-tuned model is used because this stage is extraction, not deep reasoning. On 100 held-out prompts it produced valid JSON in 94% of cases (1.8 s/spec), ahead of DeepSeek-R1-Qwen-1.5B (88.5%) and Gemma 2-2B (82%). Specifications span four domains—e-commerce, healthcare, IoT, and banking—with 500 per domain per diagram type ($4 \times 500 \times 4 = 8,000$).

Stage 2 — PlantUML synthesis (DeepSeek-R1-Distill-Qwen-32B). The JSON specification is the sole input to a 32B reasoning model with chain-of-thought prompting. Decomposition narrows the search space relative to mapping raw text directly to diagrams.

Stage 3 — Render gate and multimodal verification. PlantUML is rendered to an image. Failed renders are excluded ($\delta = 0$; composite score forced to zero). Each successful image is scored independently by three VLMs on a 0–6 rubric: semantic correctness, structural completeness, syntactic / notational accuracy, and overall coherence. Evaluators: Qwen2.5-VL-3B (MMM 53.1), LLaMA-3.2-Vision-11B (MMM 50.7), and Aya-Vision-8B (MMM 39.9). The original requirement and the JSON specification are also given to the VLMs for cross-modal consistency checking.

Dual-signal quality decision. Scores s_{ij} are combined with MMMU weights into $S_i = \delta_i \cdot (\sum w_j s_{ij}) / (\sum w_j)$, $w = (53.1, 50.7, 39.9)$. Each VLM votes accept if $s_{ij} \geq \tau = 4$; majority $A_i = 1$ if at least two of three vote yes. A sample enters the dataset only if rendering succeeds, $A_i = 1$, and $S_i \geq 3.0$. Example: scores 5, 4, 3 give $S \approx 4.09$; scores 5, 5, 4 give $S \approx 4.70$ with $A = 1$.

4. Experimental design

The study dataset contains **8,000** UML samples (2,000 per type). Human evaluation used a stratified random sample of **40 diagrams** (10 per type; seed 42). G*Power for a medium correlation effect ($f^2 = 0.15$, $\alpha = 0.05$, power 0.80) requires at least 37 observations. **Eighty** domain experts rated every sampled diagram on the same 0–6 rubric: 23 IT lecturers, 35 enterprise architects (≥ 5 years), and 22 software-engineering Ph.D. students. Inter-rater reliability uses Fleiss' Kappa; alignment between composite S and mean human scores uses Pearson r .

5. Principal results

RQ1 — generation at scale. Per-type render success is 95.7% (class), 94.4% (object), 91.6% (component), and 81.1% (package). The paper reports overall render success of **94.4%** and majority-vote acceptance of **91.3%** (6,891 of 7,553 rendered diagrams). Packages remain the bottleneck; typical failures are incorrect PlantUML nesting and renderer stress above ~50 nested elements.

Diagram	Failures	Success %	Mean S (0–6)	SD
Class	87	95.7	4.31	0.74

Object	112	94.4	4.09	0.81
Component	169	91.6	3.87	0.93
Package	379	81.1	3.12	1.04
Overall	747	94.4	3.85	0.98

Table 1. Render success and mean composite VLM score S ($n = 2,000$ per type).

RQ2 — agreement with experts. Ensemble S vs. mean human ratings: $r = 0.71$ ($p < 0.001$). Fleiss' $\kappa = 0.68$ (substantial agreement). Class diagrams align most strongly ($r = 0.82$, $\kappa = 0.74$); packages least ($r = 0.55$, $p = 0.003$, $\kappa = 0.58$), reflecting containment-versus-dependency ambiguity. Component diagrams ($r = 0.68$) diverge mainly when dense layouts impair VLM parsing.

Diagram	Pearson r	p-value	Fleiss' κ
Class	0.82	< 0.001	0.74
Object	0.76	< 0.001	0.71
Component	0.68	< 0.001	0.65
Package	0.55	$= 0.003$	0.58
Overall	0.71	< 0.001	0.68

Table 2. Automated vs. human correlation and inter-rater reliability ($n = 10$ diagrams per type; 80 evaluators).

RQ3 — why types differ. Metrics follow class > object > component > package. (1) Class diagrams dominate public training data. (2) Class PlantUML is syntactically rigid; package syntax has competing containment constructs. (3) Packages mix namespace, deployment, and grouping semantics; component boundaries must often be inferred from the requirement.

6. Comparison with prior work and contributions

A two-round search (IEEE Xplore, ACM DL, arXiv/Scholar; 187 candidates $\rightarrow$ 23 papers) identified five comparable LLM-based UML or architecture generators. This work offers (i) broader coverage (four types vs. at most two in one prior study), (ii) stronger semantic verification (three-VLM ensemble, $r = 0.71$, vs. single-VLM $r = 0.61$ in de Oliveira et al., or no automated semantic score in several prompt-only studies), and (iii) larger validated scale (8,000 vs. a prior maximum of 300). Prior limits include no automated verification (Jahan et al.; Bates et al.), image-to-code without upstream generation (Conrardy & Cabot), and Mermaid-only rendering (Gheorghita et al.).

Contributions: (1) a three-stage pipeline from requirements to design-phase UML via a structured specification; (2) an MMMU-weighted composite score with a render-failure indicator; (3) a majority-vote acceptance gate (dual-signal quality); (4) comparison with five prior approaches on render success, semantic alignment, and human correlation; (5) a public dataset of 8,000 (specification, PlantUML, score) triples.

7. Threats to validity and planned extensions

Internal. Weights come from MMMU (general multimodal reasoning), not a UML-specific benchmark. **External.** The 8,000 specifications are pipeline-generated and may miss some industrial requirement ambiguity. **Construct.** The shared 0–6 rubric's 3-versus-4 boundary was the most frequent human disagreement, especially on component diagrams. Planned extensions: sequence and activity diagrams, real OSS requirement corpora, and UML-specific VLM calibration.

8. Request to the committee member

Dr. Yutong Zhao has approved inviting you to the defense committee. I would be honored if you would consider serving. After you accept, committee members sign the DocuSign **Thesis Approval Form** when the manuscript is ready for Thesis Office submission (<https://www.csulb.edu/thesis-and-dissertation-office/signature-page-procedure>). I will send that form only after you have agreed, as advised by the graduate advisor. I am happy to share the full draft, paper, or a live pipeline demonstration on request.

Respectfully submitted for committee consideration · Dipak Yadav · Chair: Dr. Yutong Zhao · CECS, California State University, Long Beach